

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Тестирование программного обеспечения

Модуль 1. Введение в тестирование и enterprise-системы

Андрей Попов · Lektion

4 курс

Четверг 13:30–15:00

ИТМО · Тестирование ПО

План лекции

01

Что такое тестирование и зачем оно нужно

02

Enterprise-системы: чем они отличаются

03

Lekton и реальный процессинг

04

Учебный проект СМП — симулятор процессингового центра

05

Курс: 8 модулей, 8 артефактов, 50+50 баллов

06

Практика: запуск СМП

Зачем этот курс

У вас уже есть

Java

SQL

Git

REST API

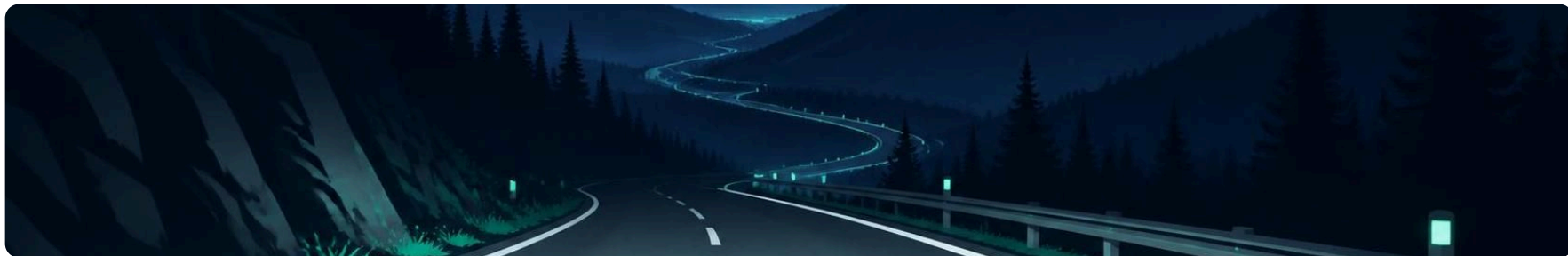
Чему научитесь

Проектировать тесты

Писать автотесты

Работать в CI/CD

Мыслить как инженер тестирования



Тестирование — оценка качества продукта через исследование его поведения.

Ошибка человека
(error)

Дефект в коде
(defect)

Отказ в работе
(failure)

Verification

«Мы делаем продукт правильно?» — соответствие спецификации

Validation

«Мы делаем правильный продукт?» — соответствие потребностям

Почему тестирование сложно

1

Exhaustive testing impossible

Невозможно проверить все комбинации входных данных

2

Показывает наличие, не отсутствие

1000 зелёных тестов $\neq$ программа без ошибок

3

Раннее тестирование

Начинайте как можно раньше
— это окупается многократно

4

Defect clustering

Большинство багов живёт в малом числе модулей

5

Парадокс пестицида

Старые тесты перестают находить новые баги

6

Цена дефекта

Удваивается на каждом уровне
— чем позже, тем дороже

Заблуждения о тестировании

✗
«Тестирование
— это кликание»

✓ Нет, это инженерная дисциплина

✗ «Автотесты
заменяют
тестировщика»

✓ Автотесты — checking, а не testing

✗
«Тестирование
гарантирует
качество»

✓ Качество строится на всех этапах

✗ «Чем больше
тестов, тем
лучше»

✓ Важна эффективность, а не количество

✗ «AI заменит
тестировщиков»

✓ AI пишет код — тестировать нужно больше

Enterprise — другой класс цены ошибки



Долгоживущий код

10–20 лет, множество интеграций, строгие требования к надёжности



Много интеграций

Платёжные системы, банки, регуляторы — каждая точка отказа



Регуляторика

PCI DSS, аудит, отчётность перед ЦБ



Цена ошибки

Дефект = финансовые потери + репутация + штрафы

Lekton и реальный процессинг

Компания

Lekton — вендор ПО для процессинговых центров

Lekton Classic — монолит

Lekton Sigma — микросервисы

~40 тестировщиков в команде

Пирамида тестирования

Unit-тесты — пишут разработчики

Модульные тесты — Java, автоматизированные

Интеграционные контура — малые контура

E2E + системное тестирование

Клиентские конфигурации



секунды

на весь путь транзакции

миллионы

транзакций в день

2–3 с

SLA на авторизацию

Учебный проект СМП

11

микросервисов

2

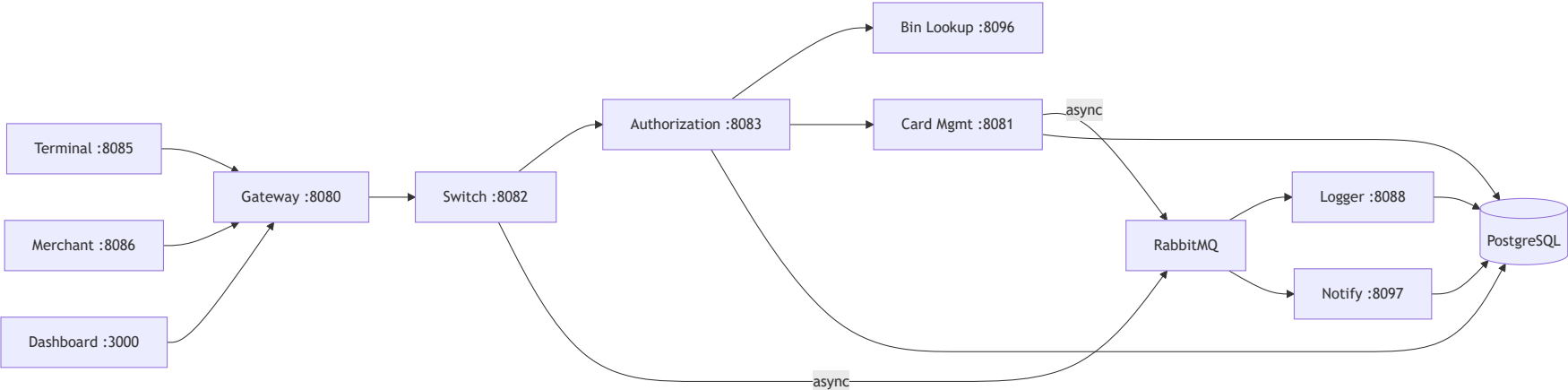
канала: HTTP +
RabbitMQ

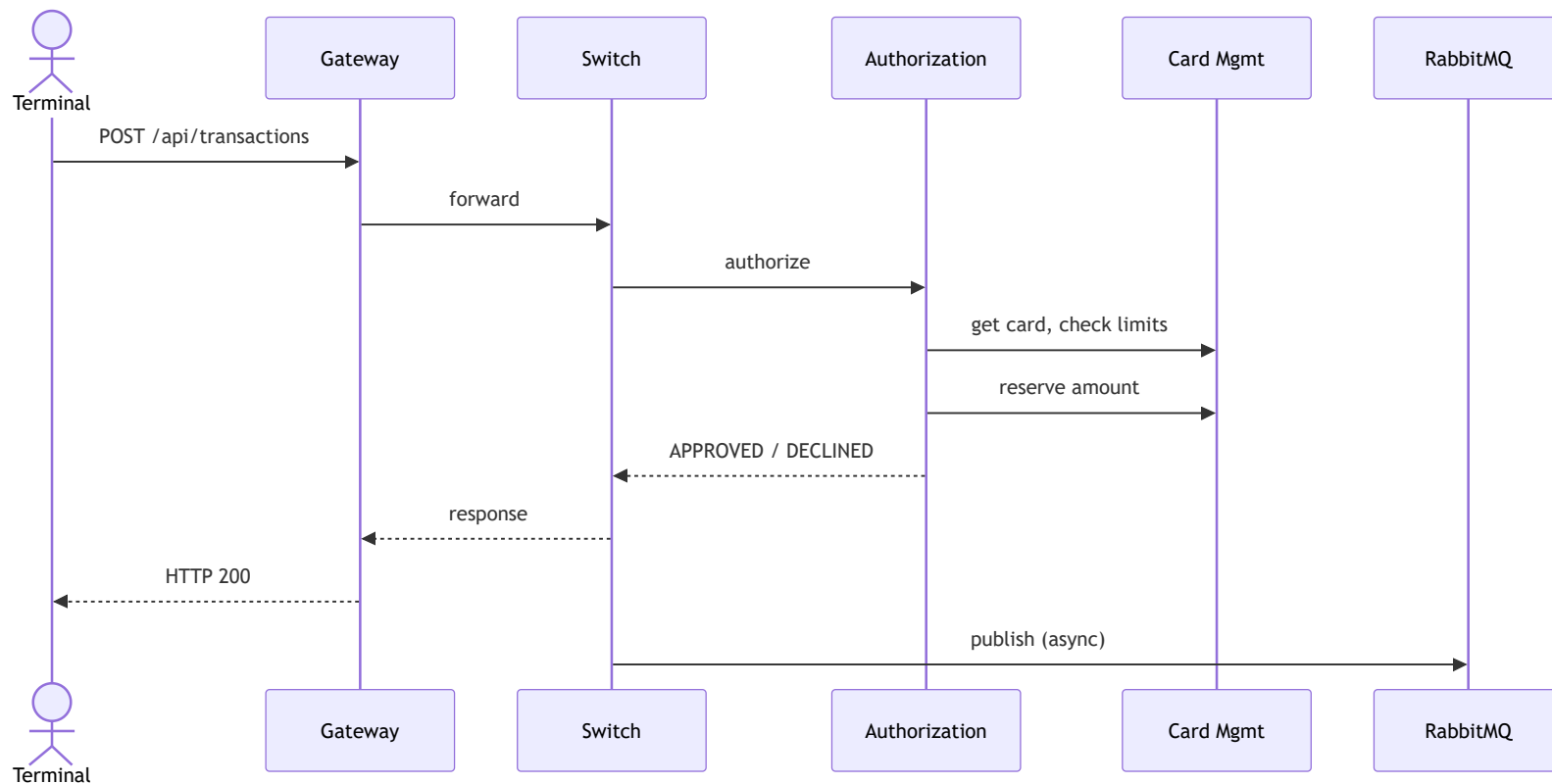
1

команда docker
compose up

0

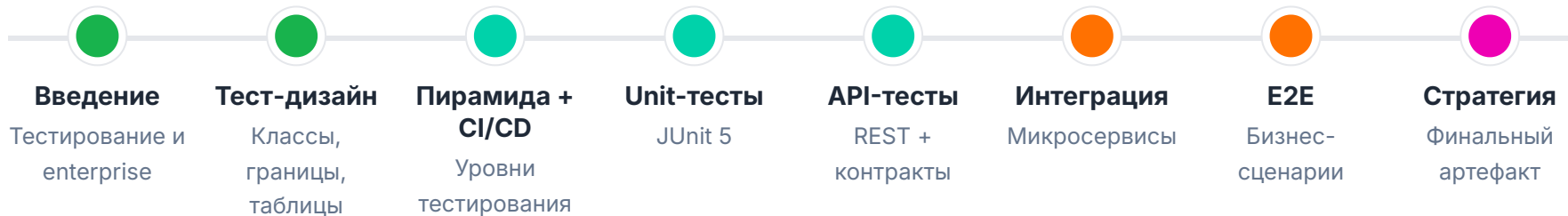
тестов — напишете
сами





⚠ Резервирование не атомарно: между GET и POST /reserve другая транзакция может изменить баланс — race condition.

Карта курса на семестр



Артефакты и баллы

№	Артефакт	Проверка	Баллы
1	Запуск СМП + smoke-тесты	CI	6
2	Тест-дизайн и баг-репорты	LLM + peer review	7
3	Unit-тесты JUnit 5	CI	7
4	API и интеграционные тесты	CI + LLM	7
5	E2E бизнес-сценарии	CI + LLM	6
6	CI/CD + нагрузочный smoke	CI + LLM	6
7	Отчётность и метрики	LLM	6
8	Финальная тестовая стратегия	Индивидуальная защита	5
Итого			50

50 баллов практика + 50 баллов посещение = 100

Что будет на практике

```
$ git clone <repo-url> smp  
$ cd smp  
$ cp .env.example .env  
$ docker compose up -d  
$ curl http://localhost:8080/health  
$ bash smoke-test.sh
```

Артефакт 1 · 6 баллов · проверка через CI